

LAPORAN PRATIUM PEMROGRAMAN ALGORITMA DAN PEMROGRAMAN
“ADT (Abstract Data Type)”



disusun Oleh:

Nama : Alhamd Ridho Putra

NIM : 2511532020

Dosen Pengampu : Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum mata kuliah Struktur Data dengan topik **Abstract Data Type (ADT)** ini dapat terselesaikan dengan baik.

Laporan ini disusun sebagai bentuk dokumentasi sekaligus pemenuhan tugas atas praktikum yang telah dilaksanakan. Melalui praktikum ini, penulis belajar mengenai pentingnya memisahkan implementasi internal data dari antarmuka penggunaannya, yang merupakan fondasi dalam pemrograman berorientasi objek dan pengembangan perangkat lunak yang kompleks.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan rekan-rekan mahasiswa lainnya.

Padang, 03 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
BAB I PENDAHULUAN.....	1
1.1. Latar Belakang.....	1
1.2. Tujuan Praktikum.....	1
1.3. Manfaat Praktikum	2
BAB II PEMBAHASAN	3
2.1. Uraian Masalah	Error! Bookmark not defined.
2.2. Langkah Kerja	Error! Bookmark not defined.
2.3. Analisis Hasil	Error! Bookmark not defined.
BAB III KESIMPULAN.....	9
3.1. Kesimpulan	9
3.2. Saran Pengembangan	9
DAFTAR PUSTAKA.....	11

DAFTAR GAMBAR

Gambar 2. 1 Kode Program Contoh Char.....	Error! Bookmark not defined.
Gambar 2. 2 Kode Program Java	Error! Bookmark not defined.
Gambar 2. 3 Kode Program Keliling Lingkaran	Error! Bookmark not defined.

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam dunia pemrograman, efisiensi dan pengorganisasian kode adalah kunci utama dalam membangun aplikasi yang handal. Seiring dengan bertambahnya kompleksitas program, pengelolaan data menjadi tantangan tersendiri.

Penggunaan tipe data dasar seperti *integer* atau *float* seringkali tidak cukup untuk merepresentasikan objek dunia nyata yang lebih kompleks.

Abstract Data Type (ADT) hadir sebagai konsep untuk mendefinisikan tipe data baru berdasarkan karakteristik logisnya, bukan pada bagaimana data tersebut disimpan dalam memori. Dengan ADT, seorang pemrogram dapat fokus pada **apa** yang dilakukan oleh data (operasi), bukan **bagaimana** cara kerjanya secara internal (*information hiding*). Hal ini memungkinkan terjadinya modularitas dan penggunaan kembali kode (*reusability*), yang sangat penting dalam kolaborasi tim pengembangan perangkat lunak.

1.2. Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ADT ini adalah:

1. **Memahami Konsep Enkapsulasi:** Mahasiswa mampu memahami cara menyembunyikan detail implementasi dari pengguna akhir.
2. **Implementasi Struktur Data:** Mahasiswa dapat merancang dan mengimplementasikan ADT (misalnya: Point, List, Stack, atau Queue) menggunakan bahasa pemrograman tertentu.
3. **Pemecahan Masalah:** Melatih kemampuan mahasiswa dalam memodelkan masalah nyata ke dalam bentuk tipe data abstrak yang sistematis.
4. **Standarisasi Kode:** Membiasakan mahasiswa untuk menulis kode yang rapi dengan pemisahan antara file header (.h) dan file implementasi (.c atau .cpp).

1.3. Manfaat Praktikum

Praktikum ini diharapkan memberikan manfaat sebagai berikut:

- **Bagi Mahasiswa:** Meningkatkan logika berpikir dalam menyusun struktur data yang efisien dan mempermudah proses *debugging* karena kode lebih terorganisir.
- **Pengembangan Perangkat Lunak:** Memberikan pemahaman dasar mengenai *Software Engineering*, di mana perubahan pada satu modul (implementasi ADT) tidak akan merusak modul lainnya selama antarmukanya tetap sama.
- **Persiapan Mata Kuliah Lanjutan:** Menjadi bekal fundamental sebelum mempelajari konsep yang lebih kompleks seperti Pemrograman Berorientasi Objek (OOP) dan Analisis Algoritma.

BAB II PEMBAHASAN

2.1 Deskripsi Abstract Data Type (ADT) Jam

Pada praktikum ini, implementasi ADT diwujudkan melalui kelas `Jam_2511532020`. Kelas ini bertindak sebagai tipe data abstrak yang merepresentasikan waktu dalam format Jam, Menit, dan Detik (HH:MM:SS). Sesuai dengan prinsip ADT, kelas ini menyembunyikan detail representasi data (menggunakan akses *private* pada atribut) dan menyediakan antarmuka (*public methods*) untuk berinteraksi dengan data tersebut.

2.2 Uraian Kode Program

Kode program dibagi menjadi tiga bagian utama, yaitu pendefinisian ADT dan dua buah *driver* sebagai penguji.

2.2.1 Kelas ADT `Jam_2511532020`

```
package pekan1_2511532020;

public final class Jam_2511532020 {
    private int hh;
    private int mm;
    private int ss;

    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s
&& s <= 59);
    }

    public Jam_2511532020(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    public void setHour(int h) {this.hh = h; }
    public void setMinute(int m) {this.mm = m; }
    public void setSecond(int s) {this.ss = s; }
```

```

    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511532020 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik
negatif");
        total %= 24 * 3600;
        int h = total / 3600;
        int m = (total % 3600) / 60;
        int s = total % 60;
        return new Jam_2511532020(h, m, s);
    }

    public int compareTo(Jam_2511532020 other) { return
Integer.compare(this.toSeconds(), other.toSeconds());}
    public boolean equals (Object o) {
        if (!(o instanceof Jam_2511532020 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }

    public Jam_2511532020 plus(Jam_2511532020 other) { return
fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511532020 minus(Jam_2511532020 other) { return
fromSeconds(this.toSeconds() - other.toSeconds()); }
    public Jam_2511532020 nextSecond() { return
fromSeconds(this.toSeconds() + 1); }
    public Jam_2511532020 nextNseconds(int n) { return
fromSeconds(this.toSeconds() + Math.max(0, n)); }
    public Jam_2511532020 prevSecond() { return
fromSeconds(Math.floorMod(this.toSeconds()-1, 24 * 3600)); }
    public Jam_2511532020 prevNSecond(int n) { return
fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 *
3600)); }

    public static int durasiDetik(Jam_2511532020 jaw, Jam_2511532020
jakh) { return jakh.toSeconds() - jaw.toSeconds(); }

    public String toString() { return String.format("%02d:%02d:%02d",
hh, mm, ss); }

}

```


Kelas ini merupakan inti dari praktikum yang mencakup beberapa komponen penting:

1. Atribut (State): Menggunakan tiga variabel *private* hh (jam), mm (menit), dan ss (detik) untuk menerapkan prinsip enkapsulasi.
2. Validasi: Method *isValid* bersifat *static*, berfungsi untuk memastikan bahwa input jam berada pada rentang 0-23, serta menit dan detik pada rentang 0-59 sebelum objek dibuat.
3. Konversi: Method *toSeconds()* dan *fromSeconds()* berfungsi untuk mengubah format jam menjadi total detik dan sebaliknya. Ini adalah teknik krusial untuk memudahkan operasi aritmatika waktu.
4. Operasi Aritmatika: Terdapat fungsi plus, minus, *nextSecond*, dan *prevSecond*. Penggunaan *Math.floorMod* pada operasi *previous* memastikan bahwa jika waktu mundur melewati pukul 00:00:00, maka waktu akan berputar kembali ke 23:59:59 (siklus 24 jam).
5. Perbandingan: Method *compareTo* dan *equals* digunakan untuk membandingkan dua objek jam berdasarkan nilai total detiknya.

2.2.2 Kelas Driver (JamDriver & JamDriver2)

```
package pekan1_2511532020;
```

```
public class JamDriver_2511532020 {

    public static void main(String[] args) {
        Jam_2511532020 a = new Jam_2511532020(23, 59, 50);
        Jam_2511532020 b = new Jam_2511532020(0, 0, 15);
        System.out.println("a          = " + a);
        System.out.println("b          = " + b);
        System.out.println("a+b      = " + a.plus(b));
        System.out.println("next 20s  = " +
a.nextNseconds(20));
        System.out.println("durasi(a,b) = " +
Jam_2511532020.durasiDetik(a, b));
        System.out.println("a.compareTo(b) = " +
a.compareTo(b));

    }

}
```

- JamDriver_2511532020: Berfungsi sebagai pengujian dasar (*hardcoded*) untuk memvalidasi apakah logika penambahan waktu dan perhitungan durasi sudah berjalan benar.

```
package pekan1_2511532020;

import java.util.Scanner;

public class JamDriver2_2511532020 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam
===");

        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511532020 j1 = buatJamdariInput (input);

        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511532020 j2 = buatJamdariInput (input);

        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)      : " +
j1.toString());
        System.out.println("Jam 2 (String)      : " +
j2.toString());
        System.out.println("Jam 1 dalam detik   : " +
j1.toSeconds());
        System.out.println("Jam 2 dalam detik   : " +
j2.toSeconds());

        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status          : Jam 1
lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status          : Jam 1
lebih awal (sebelum) Jam 2");
        } else {
            System.out.println("Status          : Jam 1
dan Jam 2 sama persis");
        }

        System.out.println("Durasi (J1 ke J2)    : " +
Jam_2511532020.durasiDetik(j1, j2));

        Jam_2511532020 jNext = j1.nextSecond();
        System.out.println("Jam 1 detik berikutnya : " +
jNext);

        Jam_2511532020 jPrev = j1.prevSecond();
        System.out.println("Jam 1 detik sebelumnya : " +
jPrev);
    }
}
```

```

        Jam_2511532020 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2      : " +
jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");

    }

    private static Jam_2511532020 buatJamdariInput
(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23)
: ");

            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59)
: ");

            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59)
: ");

            s = sc.nextInt();

            if (Jam_2511532020.isValid(h, m, s)) {
                return new Jam_2511532020(h, m,
s);
            } else {
                System.out.println("[Error]
Input tidak valid! silahkan ulangi.\n");
            }
        }
    }
}

```

- JamDriver2_2511532020: Berfungsi sebagai antarmuka interaktif. Di sini diterapkan *error handling* menggunakan perulangan while(true) dan pengecekan isValid(). Jika pengguna memasukkan angka di luar nalar (misal: jam 25), program akan menolak dan meminta input ulang.

2.3 Penjelasan Langkah Kerja

Proses eksekusi program mengikuti alur logika sebagai berikut:

1. Instansiasi: Program meminta input jam, menit, dan detik dari pengguna.
2. Validasi: Sebelum objek dikonstruksi, data diperiksa kelayakannya. Jika valid, objek j1 dan j2 dibuat di dalam memori.
3. Proses Aritmatika: * Untuk operasi plus atau nextSecond, program mengubah waktu saat ini menjadi total detik, menambahkannya, lalu

mengubahnya kembali menjadi format Jam-Menit-Detik menggunakan fungsi `fromSeconds`.

- Untuk menghitung selisih (durasi), program mengurangi total detik jam akhir dengan jam awal.
4. Output: Hasil diubah kembali menjadi tipe String melalui *override* method `toString()` dengan format `%02d` agar angka satuan tetap memiliki nol di depannya (contoh: 01:05:09).

2.4 Analisis Hasil dan Teori

Berdasarkan hasil eksekusi pada `JamDriver2`, diperoleh beberapa analisis penting:

- Penanganan *Overflow*: Saat jam 23:59:50 ditambah 20 detik, hasil yang diperoleh adalah 00:00:10. Hal ini membuktikan bahwa logika Modular Arithmetic ($\text{total} \% (24 * 3600)$) berhasil menangani pergantian hari secara otomatis.
- Independensi Data: Pengguna *driver* tidak perlu tahu bagaimana cara detik dihitung secara internal. Mereka hanya memanggil method `plus()` atau `nextSecond()`. Inilah yang disebut dengan Information Hiding dalam teori Struktur Data.
- Konsistensi Objek: Penggunaan kata kunci final pada kelas dan *immutable-like behavior* (di mana operasi tambah/kurang menghasilkan objek baru bukannya mengubah objek lama) menjaga integritas data selama program berjalan.

BAB III

KESIMPULAN

3.1. Kesimpulan

Berdasarkan hasil praktikum dan pembahasan yang telah dilakukan pada implementasi ADT Jam_2511532020, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Efektivitas Enkapsulasi:** Penerapan hak akses *private* pada atribut hh, mm, dan ss terbukti efektif dalam melindungi integritas data. Perubahan nilai hanya dapat dilakukan melalui mekanisme validasi yang ketat, sehingga mencegah terbentuknya objek jam yang tidak logis (misal: menit bernilai 70).
2. **Kemudahan Operasi melalui Konversi:** Penggunaan metode antara (*intermediate*) seperti toSeconds() sangat menyederhanakan logika aritmatika waktu. Dengan mengubah waktu menjadi total detik, operasi penjumlahan, pengurangan, dan perbandingan dapat dilakukan dengan operasi matematika dasar sebelum dikonversi kembali ke format jam.
3. **Modularitas dan Reusabilitas:** Konsep ADT memungkinkan pemisahan yang jelas antara logika data (kelas Jam) dan logika aplikasi (kelas Driver). Hal ini memudahkan pengujian kode tanpa harus mengubah struktur dasar tipe datanya.
4. **Penanganan Siklus Waktu:** Penggunaan operator modulus (%) dan Math.floorMod berhasil menangani masalah *overflow* dan *underflow* pada sistem waktu 24 jam, memastikan transisi antar hari (seperti dari 23:59:59 ke 00:00:00) berjalan secara akurat.

3.2. Saran Pengembangan

Program ini masih memiliki potensi untuk dikembangkan lebih lanjut agar lebih kompleks dan fungsional. Beberapa saran pengembangan yang dapat dilakukan adalah:

1. **Integrasi Penanggalan (Date):** Mengembangkan ADT ini menjadi `DateTime` sehingga tidak hanya mengelola waktu dalam satu hari (24 jam), tetapi juga dapat menangani pergantian hari, bulan, hingga tahun.
2. **Implementasi Interface Standard:** Mengimplementasikan *interface* `Comparable<Jam>` bawaan Java secara formal agar objek Jam dapat langsung diurutkan menggunakan fungsi standar seperti `Collections.sort()`.
3. **Penambahan Satuan Milidetik:** Untuk kebutuhan sistem yang memerlukan presisi tinggi (seperti aplikasi pencatat waktu olahraga), program dapat dikembangkan dengan menambahkan atribut milidetik (*ms*).
4. **Antarmuka Grafis (GUI):** Mengembangkan *driver* aplikasi menggunakan JavaFX atau Swing agar pengguna dapat berinteraksi dengan objek Jam melalui jam analog atau digital secara visual, bukan hanya melalui terminal teks.
5. **Fitur Zona Waktu:** Menambahkan parameter zona waktu (GMT/UTC) agar program dapat melakukan konversi waktu antar wilayah secara otomatis.

TUGAS PRAKTIKUM

1. KODE ADT

```
package pekan1_2511532020;

public class Mobil_2511532020 {
    // Atribut (Private sesuai prinsip Enkapsulasi ADT)
    private String nama;
    private int tahun;
    private int cc;
    private long harga; // Menggunakan long karena harga mobil biasanya
    // besar
    private String merk;

    // --- Constructor (Representasi dari "Tambah Mobil") ---
    public Mobil_2511532020(String nama, int tahun, int cc, long harga,
    String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    // --- Selektor (Getters) ---
    public String getNama() { return nama; }
    public int getTahun() { return tahun; }
    public int getCc() { return cc; }
    public long getHarga() { return harga; }
    public String getMerk() { return merk; }

    // --- Mutator (Setters) ---
    public void setNama(String nama) { this.nama = nama; }
    public void setTahun(int tahun) { this.tahun = tahun; }
    public void setCc(int cc) { this.cc = cc; }
    public void setHarga(long harga) { this.harga = harga; }
    public void setMerk(String merk) { this.merk = merk; }

    // --- Method Hapus Mobil ---
    // Dalam konteks ADT sederhana, menghapus berarti mengosongkan nilai
    // atribut objek
    public void hapusMobil() {
        this.nama = "-";
        this.tahun = 0;
        this.cc = 0;
        this.harga = 0;
        this.merk = "-";
        System.out.println("Data mobil telah dihapus/direset.");
    }

    // Method pembantu untuk menampilkan data
    @Override
    public String toString() {
        return String.format("Mobil [Merk: %s, Nama: %s, Tahun: %d, CC:
        %d, Harga: Rp%,d]",
                                merk, nama, tahun, cc, harga);
    }
}
```

```
}  
}
```

2. KELAS

```
package pekan1_2511532020;  
  
public class MobilDriver_2511532020 {  
    public static void main(String[] args) {  
        System.out.println("=== Program ADT Mobil - NIM 2511532020  
===");  
  
        // 1. Tambah Mobil (Membuat Objek Baru)  
        Mobil_2511532020 mobil1 = new Mobil_2511532020("Avanza", 2020,  
1300, 150000000, "Toyota");  
  
        // 2. Menampilkan Data (Menggunakan Selektor)  
        System.out.println("\nData Mobil Saat Ini:");  
        System.out.println(mobil1.toString());  
  
        // 3. Mengubah Data (Menggunakan Mutator)  
        System.out.println("\n--- Mengubah Harga dan Tahun Mobil ---");  
        mobil1.setHarga(160000000);  
        mobil1.setTahun(2021);  
  
        System.out.println("Data setelah update: " + mobil1.getNama() +  
" (" + mobil1.getTahun() + ")");  
        System.out.println("Harga terbaru: Rp" + mobil1.getHarga());  
  
        // 4. Hapus Mobil  
        System.out.println("\n--- Menghapus Data Mobil ---");  
        mobil1.hapusMobil();  
        System.out.println("Status data sekarang: " +  
mobil1.toString());  
    }  
}
```


DAFTAR PUSTAKA

- [1] P. J. Deitel and H. M. Deitel, *Java: How to Program (Early Objects)*, 11th ed. New York, NY, USA: Pearson, 2017.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*. Hoboken, NJ, USA: John Wiley & Sons, 2014.
- [3] H. Schildt, *Java: The Complete Reference*, 11th ed. New York, NY, USA: McGraw-Hill Education, 2018.
- [4] M. A. Weiss, *Data Structures and Problem Solving Using Java*. Boston, MA, USA: Pearson, 2012.
- [5] R. Munir, *Algoritma dan Pemrograman dalam Bahasa Pascal dan C*. Bandung, Indonesia: Informatika, 2011.
- [6] I. Liem, *Diktat Struktur Data*. Bandung, Indonesia: Teknik Informatika, Institut Teknologi Bandung, 2003.
- [7] S. Horwitz, "Abstract Data Types and Object-Oriented Programming," Computer Sciences Department, University of Wisconsin-Madison, 2023.